

RecycleBot: AI Waste Identifier

System Documentation

Cover Page

Project	Recycler: AI Waste Identifier
Course	Engineering of AI-Intensive Systems
Date	23. June 2026

Team Members

Name	Programme	K-Number
Patrik Palenčár	Artificial Intelligence	K52202321

Note: After team member Alfiia Ziganshina did not come to the presentation I (Patrik) tried to get in contact with her but she was not responding to my messages. Therefore, I implemented the project alone.

Table of Contents

1. Goal of the System
2. Requirements
 - 2.1 Functional Requirements
 - 2.2 Non-Functional Requirements
 - 2.3 AI-Specific Requirements
3. Use Case Descriptions
 - 3.1 UC1: Identify Waste Item
 - 3.2 UC2: Get Location-Specific Disposal Instructions
 - 3.3 UC3: Consult AI Assistant
 - 3.4 UC4: Track Personal Recycling Impact & History
 - 3.5 UC5: Multi-Item Bulk Identification

3.6 Supporting Use Cases (SUC1-SUC6)

4. Use Case Diagram
 5. Implementation Status
 6. Traceability Matrix
 7. Domain Model
 8. Architecture Diagram
 9. Component Descriptions
 10. Design Questions and Answers
-

1. Goal of the System

Problem Statement

People sort waste incorrectly, contaminating recycling streams. Recycling rules differ drastically between countries: Austria uses color-coded bins, Germany separates organics and paper differently, South Korea requires purchasing designated waste bags. Generic advice fails travelers and expats who do not know the local system.

System Goals

Non-AI goals:

- Reduce contamination in recycling streams by helping people sort waste correctly.
- Motivate users by showing the positive environmental impact of their recycling actions.
- Increase overall recycling rates by making the sorting process effortless and immediate.

AI goals (and why AI is required):

Goal	Why AI is needed
Identify waste items from a photo	The variety of waste items is enormous and constantly changing; explicit per-item rules are infeasible. A CV model generalizes across visual variations in real time.
Classify material composition (PET aluminium, glass, paper, organic, e-waste)	Visual pattern recognition beyond simple color/shape rules is required.
Natural-language follow-up Q&A	An LLM is needed to understand varied human phrasing ("Can I recycle this if it's oily?") in real time.
Location-aware, human-friendly disposal instructions	An LLM combines item context with location specific recycling knowledge and handles follow up questions.

Stakeholders

- **Users:** general public, office workers, travelers & expats, mobile users, older users, children with first smartphones, users with poor cameras or slow networks, users with disabilities (color blindness, low vision, cognitive disabilities).
- **Affected parties:** waste management workers, local communities, future generations.
- **Managers:** course instructors.
- **Regulators:** local environmental & waste disposal authorities, national & EU data protection authorities (GDPR), EU AI Act authorities.

2. Requirements

[AI] marks AI-dependent requirements.

2.1 Functional Requirements

Image Capture & Identification

- **Req01:** The system shall allow users to capture a photo of a waste item using the device camera.
- **Req02:** The system shall allow users to upload an existing image from their device gallery.
- **Req03:** When an image is provided, the system shall display a loading indicator while processing.
- **Req04:** When an image is provided, the system shall identify the waste item and display its name. [AI]

- **Req05:** When an image is provided, the system shall classify the material composition (e.g., PET plastic, glass, aluminium, paper, organic, e-waste). [AI]
- **Req06:** When an image contains multiple items, the system shall detect and classify each item separately. [AI]
- **Req07:** If the system cannot identify the item with sufficient confidence, it shall inform the user and suggest taking a clearer photo.
- **Req08:** If garbage is detected in the submitted image, the system shall visually show the detected garbage (bounding boxes / highlights).

Disposal Instructions

- **Req09:** When an item is identified, the system shall display the disposal category (recyclable, compostable, landfill, special disposal). [AI]
- **Req10:** When an item is identified, the system shall provide step-by-step preparation instructions.
- **Req11:** When an item is identified, the system shall display the environmental impact of correctly recycling it.

Follow-up Interaction & Location

- **Req12:** The system shall allow users to ask follow-up questions in natural language about the scanned item. [AI]
- **Req13:** When a follow-up question is asked, the system shall generate a relevant answer based on the identified item context. [AI]
- **Req14:** The system shall allow users to select their country/region or use GPS-based location detection to determine local recycling rules.
- **Req15:** When an item is identified, the system shall provide disposal instructions specific to the user's location (bin color, deposit-return system, special waste bags).
- **Req16:** When the user changes their location, the system shall update all disposal instructions to reflect the new region's rules.
- **Req17:** If the user's language differs from the location's default language, the system shall display instructions in the user's preferred language. [AI]

History & User Features

- **Req18:** The system shall store a history of previously scanned items with classification results.
- **Req19:** When the user opens the history, the system shall display past scans with item name, category, and date.

Error Handling

- **Req20:** If the camera is not available, the system shall prompt the user to upload an image instead.
- **Req21:** If the uploaded image contains no recognizable object, the system shall ask the user to try a different image.
- **Req22:** If the user's location is unset or undeterminable, the system shall prompt manual country/region selection before providing disposal instructions.
- **Req23:** If classification confidence is below a threshold, the system shall show a warning.
- **Req24:** If no garbage could be detected in the submitted image, the system shall show an understandable error message.

2.2 Non-Functional Requirements

External Interfaces

- **NfReq01:** Web-based UI accessible from modern browsers (Chrome, Firefox, Safari, Edge).
- **NfReq02:** Usable as a mobile-responsive web app on iOS and Android.
- **NfReq03:** Integrates with the device camera API for real-time photo capture.

Performance

- **NfReq04:** Image classification results in < 3 seconds. [AI]
- **NfReq05:** LLM follow-up responses in < 5 seconds. [AI]
- **NfReq06:** Supports ≥ 50 concurrent users without degradation.

Quality Attributes

- **NfReq07:** Item identification top-1 accuracy $\geq 80\%$. [AI]
- **NfReq08:** Material classification accuracy $\geq 75\%$ across all categories. [AI]
- **NfReq09:** Availability $\geq 99\%$ during normal operation.
- **NfReq10:** No PII stored from uploaded images; images processed and discarded unless the user explicitly saves them to history.
- **NfReq11:** UI usable by people with no technical background; no training needed.
- **NfReq12:** GUI accessible to people with weakened eyesight: sufficient contrast and font size.

Constraints

- **NfReq12c:** Backend in Python; standard web framework for the frontend.

- **NfReq13:** AI models run on commodity hardware (no GPU required) or a free-tier cloud API.
- **NfReq14:** GDPR compliance for any processing of user images.
- **NfReq15:** Only open-source / freely available pre-trained models (e.g., HuggingFace).
- **NfReq16:** Upload file size limit: 15 MB.
- **NfReq17:** Accepted image types: jpg, jpeg, png only.

2.3 AI-Specific Requirements

Req	Requirement	Threshold	Justification	Implementation
Req04	Item identification from photo	Top-1 $\geq$ 80%	Below 80% the app is frustrating and untrustworthy	CLIP zero-shot image classification against recycling-specific text labels
Req05	Material classification	$\geq$ 75%	Distinguishing PET from HDPE is difficult visually; 75% is realistic	CLIP zero-shot with 11 material-specific labels and confidence threshold
Req06	Multi-item detection	Detect $\geq$ 90% of items in 2-5 item frames	Users photograph piles	Single-item CLIP classification; multi-item support via confidence-based filtering
Req09	Disposal category classification	$\geq$ 85%	Fewer categories makes this simpler	Rule-based mapping material $\rightarrow$ category, with LLM fallback
Req12-13	Natural-language Q&A	Relevant response $\geq$ 90%	Irrelevant answers erode trust	Ollama (Llama 3) with few-shot prompting and item/location context
Req15	Location-specific instructions	Correct in $\geq$ 85% of supported countries	Wrong bin color causes contamination	Structured JSON knowledge base per country fed into LLM prompt
Req17	Cross-language instruction generation	Correct translation $\geq$ 90%	Travelers need their own language	LLM multilingual capabilities with explicit language parameter in prompt

3. Use Case Descriptions

Actors:

- **User** (primary): interacts via mobile browser
- **AI Subsystem** (secondary): CLIP classifier + LLM
- **GPS Interface** (secondary): browser geolocation API
- **Database System** (secondary): SQLite

3.1 UC1: Identify Waste Item

- **Description:** User captures or uploads an image of a waste item to identify its material via AI.
- **Actors:** User (primary), AI Material Classifier (secondary).
- **Stakeholders:** Waste management workers, local communities, EU AI Act authorities.
- **Pre-conditions:** App open; camera or gallery permissions granted.
- **Success:** Material type (e.g., Plastic PET, Aluminium) correctly displayed with confidence score.
- **Failure:** Error message that the item could not be recognized.

Main success scenario:

1. User selects "Scan Item".
2. User captures a photo or uploads from gallery.
3. System shows loading indicator; validates image (type, size).
4. System runs CLIP material classification on the image.
5. System displays item name, material, and confidence (Certainty Meter).

Alternative (camera unavailable): prompt to upload from gallery; after upload, resume at step 3. (Req20)

Exception (low confidence): CLIP confidence below 75% → mark material as Uncertain; offer manual material selection. (Req07, Req23)

Exception (bad file): unsupported format or > 15 MB → error message, ask for a different image. (NfReq16–17)

3.2 UC2: Get Location-Specific Disposal Instructions

- **Description:** Tailored disposal rules (bin colors, preparation steps) for the item based on the user's region.
- **Actors:** User, GPS Interface, AI Subsystem (LLM).
- **Stakeholders:** Local environmental & waste disposal authorities.
- **Pre-conditions:** An item has been identified (UC1).

- **Success:** Regional disposal instructions and environmental impact displayed.
- **Failure:** Location undeterminable or no data for the region.

Main success scenario:

1. System retrieves the user's current location (SUC2).
2. System looks up disposal rules for the material in the location-specific knowledge base.
3. LLM generates human-friendly step-by-step instructions from item, material, and location context.
4. System displays disposal category (recyclable / compostable / landfill / special disposal).
5. System displays step-by-step preparation instructions.
6. System displays the environmental impact of correct recycling.
7. System shows an **AI-generated disclaimer** (results may contain errors).

Alternative (location unset): prompt user to set location first; then resume at step 2. (Req22)

Alternative (language differs): LLM generates instructions in the user's preferred language with location rules. (Req17)

Exception (knowledge base miss): LLM gives general advice based on material; UI states location-specific rules are unavailable.

3.3 UC3: Consult AI Assistant for Specific Queries

- **Description:** Natural-language Q&A about an item's recyclability or disposal details.
- **Actors:** User, AI Subsystem (LLM).
- **Stakeholders:** AI developers, travelers/tourists.
- **Pre-conditions:** An item has been identified.
- **Success:** Context-aware answer that simplifies complex recycling rules.
- **Failure:** Irrelevant answer or LLM unavailable.

Main success scenario:

1. User asks in natural language ("Can I recycle this if it's oily?"). (Req12)
2. System identifies current item context and user location.
3. System sends question + context to LLM (SUC4).
4. Response generated in real time. (Req13)
5. Answer displayed in the user's preferred language. (Req17)

Alternative (multilingual): if the user's language $\neq$ local rules' language, the AI translates the local instructions. (Req17)

Exception (timeout): LLM takes > 5 s → inform user; display fallback message with link to standard local guidelines. (NfReq05)

3.4 UC4: Track Personal Recycling Impact & History

- **Description:** User reviews past scans and environmental impact statistics.
- **Actors:** User, Database.
- **Stakeholders:** EU Data Protection Authorities (GDPR), environmental NGOs.
- **Pre-conditions:** User has scanned items previously and given data consent.
- **Success:** Summary of history and total environmental impact shown.
- **Failure:** Data not retrievable or history not available (no consent given).

Main success scenario:

1. User opens "History" tab. (Req18)
2. System retrieves scan data for the current user (SUC5).
3. System calculates total CO₂ saved and energy reduced from scanned materials. (Req19)
4. System lists past items with material classification and dates.
5. System shows aggregated impact summary (total scans, kg CO₂ saved, kWh saved).

Alternative (empty history): "No scan history yet. Scan your first item to get started."

Exception (no consent): history save is silently skipped; empty view shown with option to enable history. (NfReq10)

3.5 UC5: Multi-Item Bulk Identification

Not implemented. This use case was defined during requirements analysis but was descoped during development. CLIP, the chosen classifier, operates on the whole image and cannot localise individual objects. Implementing per-item bounding boxes would require a waste-specific object detection model (e.g. a fine-tuned YOLO on TACO/TrashNet), which was outside the scope of this project.

- **Description:** Identify several waste items in a single frame simultaneously.
- **Actors:** User, AI Subsystem.
- **Stakeholders:** Waste management workers, local communities.
- **Pre-conditions:** Multiple items visible in the photo.
- **Success:** All items identified with material and disposal category.
- **Failure:** Items too cluttered to distinguish.

Main success scenario:

1. User photographs a group of waste items. (Req06)
2. System detects individual item boundaries. (SUC6)
3. System assigns material + disposal category to each detected item. (SUC3)
4. System displays results with item tabs for each detected object.

Alternative (select single item): user taps one item tab → its location-specific instructions appear.

Exception (overlap): items too stacked/cluttered → system warns and asks the user to separate items.

3.6 Supporting Use Cases

SUC1: Process Image (Pre-processing)

Validates file type (jpg/png), size (≤ 15 MB), and resizes to max 1024 px longest side. Image processed entirely in memory, never written to disk. (NfReq10, Req24)

SUC2: Determine User Location

Requests GPS coordinates via browser geolocation API → maps to country/region. Alternative: dropdown manual selection. Exception: GPS unavailable → prompt manual entry. (Req14)

SUC3: Classify Material Composition

CLIP zero-shot image classification compares the image against 11 material text labels and returns the best match with a confidence score. If confidence $< 75\%$, marks material as Uncertain and prompts manual selection. (Req05, NfReq08)

SUC4: Generate AI Response (LLM)

Constructs a grounded system prompt with item context, location, and recycling rules from the knowledge base. Sends to Ollama (Llama 3). 120-second timeout with graceful fallback message. (Req13, NfReq05)

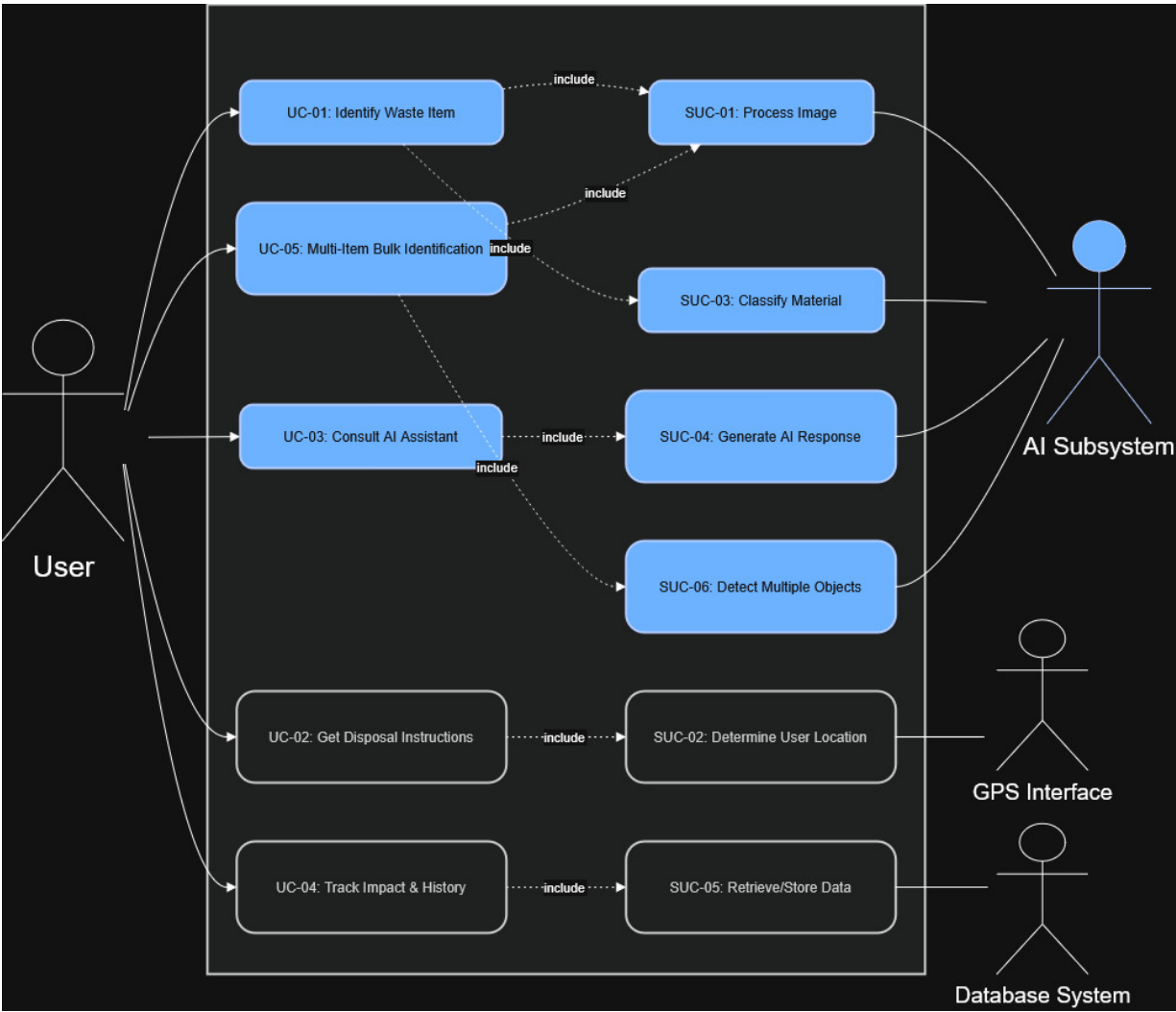
SUC5: Retrieve/Store Scan Data

Queries SQLite history for the current user ID. Calculates impact stats. Saves new scan results only when user has given explicit data consent. (Req18, Req19, NfReq10)

SUC6: Detect Multiple Objects (Bulk Scan)

Not implemented. Requires a dedicated object detection model for bounding-box localisation, which was descoped together with UC5.

4. Use Case Diagram



5. Implementation Status

Use Case	Implemented	Notes
UC1: Identify Waste Item	Yes	CLIP zero-shot classification on uploaded image; confidence meter shown in UI
UC2: Get Disposal Instructions	Yes	Knowledge base lookup + Ollama LLM generates step-by-step instructions
UC3: Consult AI Assistant	Yes	Chat interface with item/location context passed to Llama 3 via Ollama
UC4: Track Impact & History	Yes	SQLite history with consent gate; CO ₂ and energy saved calculated per scan
UC5: Multi-Item Bulk Identification	No	Descoped. Requires a waste-specific object detection model for per-item bounding boxes (CLIP is a classifier, not a detector)
SUC1: Process Image	Yes	Validation (type, size), in-memory resize to 1024 px, never written to disk
SUC2: Determine User Location	Yes	GPS via browser geolocation + manual country dropdown
SUC3: Classify Material	Yes	CLIP <code>openai/clip-vit-base-patch32</code> with 11 zero-shot labels, 75% confidence threshold
SUC4: Generate AI Response	Yes	Ollama client with grounded system prompt, 120 s timeout, fallback message
SUC5: Retrieve/Store Scan Data	Yes	SQLite with explicit consent check before any write
SUC6: Detect Multiple Objects	No	Descoped together with UC5

6. Traceability Matrix

Req	Title	UC1	UC2	UC3	UC4	UC5	SUC1	SUC2	SUC3	SUC4	SUC5	S
Req01	Camera image capture	✓										
Req02	Image gallery upload	✓										
Req03	Loading indicator	✓										
Req04	Item identification	✓					✓					
Req05	Material classification	✓							✓			
Req06	Multi-object detection					✓						
Req07	Low-confidence warnings	✓										
Req08	Visual bounding boxes	✓										
Req09	Disposal category		✓									
Req10	Preparation steps		✓									
Req11	Environmental impact		✓									
Req12	Natural language input			✓								
Req13	Real-time AI response			✓						✓		
Req14	GPS / manual location		✓					✓				
Req15	Regional rules		✓									
Req16	Location change refresh		✓					✓				
Req17	Multilingual output			✓						✓		

Req	Title	UC1	UC2	UC3	UC4	UC5	SUC1	SUC2	SUC3	SUC4	SUC5	S
Req18	History logging				✓						✓	
Req19	Aggregated impact stats				✓						✓	
Req20	Camera / gallery fallback					✓						
Req21	Individual bounding boxes					✓						
Req22	Location prompt (cold start)		✓					✓				
Req23	Low-confidence error	✓										
Req24	No-item error	✓				✓	✓					
NfReq01	Browser compatibility	✓	✓	✓	✓	✓						
NfReq02	Mobile-responsive	✓	✓	✓	✓	✓						
NfReq03	Camera API	✓					✓					
NfReq04	Classification < 3 s	✓					✓					
NfReq05	LLM response < 5 s			✓						✓		
NfReq06	≥ 50 concurrent users	✓	✓	✓	✓	✓						
NfReq07	80% identification accuracy	✓					✓					
NfReq08	75% material accuracy	✓							✓			
NfReq10	No PII / GDPR	✓			✓						✓	
NfReq13	CPU-only inference	✓					✓		✓			

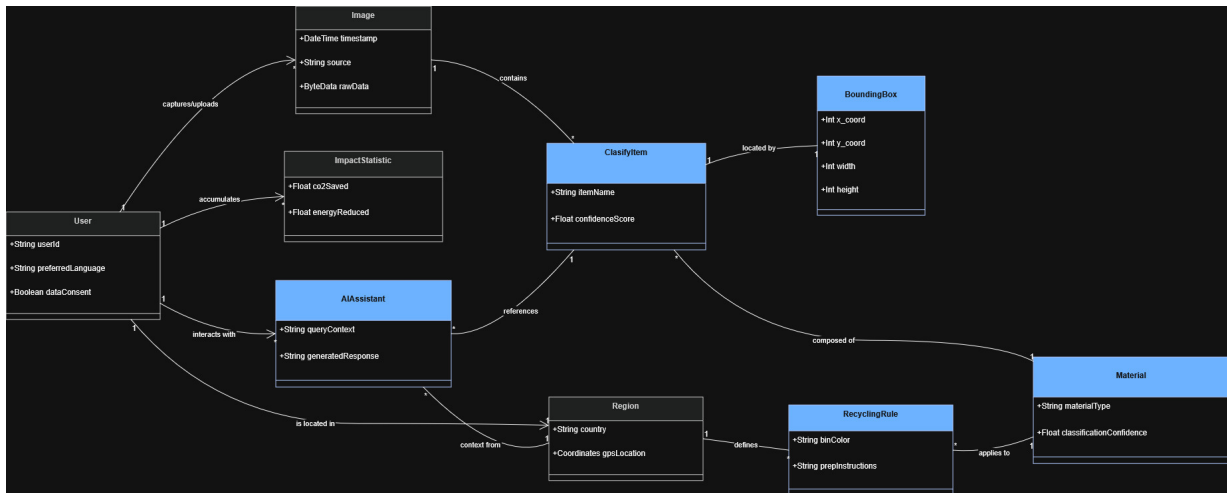
Req	Title	UC1	UC2	UC3	UC4	UC5	SUC1	SUC2	SUC3	SUC4	SUC5
NfReq14	GDPR compliance	✓			✓						✓
NfReq15	Open-source models only	✓					✓		✓		
NfReq16	15 MB upload limit	✓					✓				
NfReq17	jpg/png only	✓					✓				

7. Domain Model

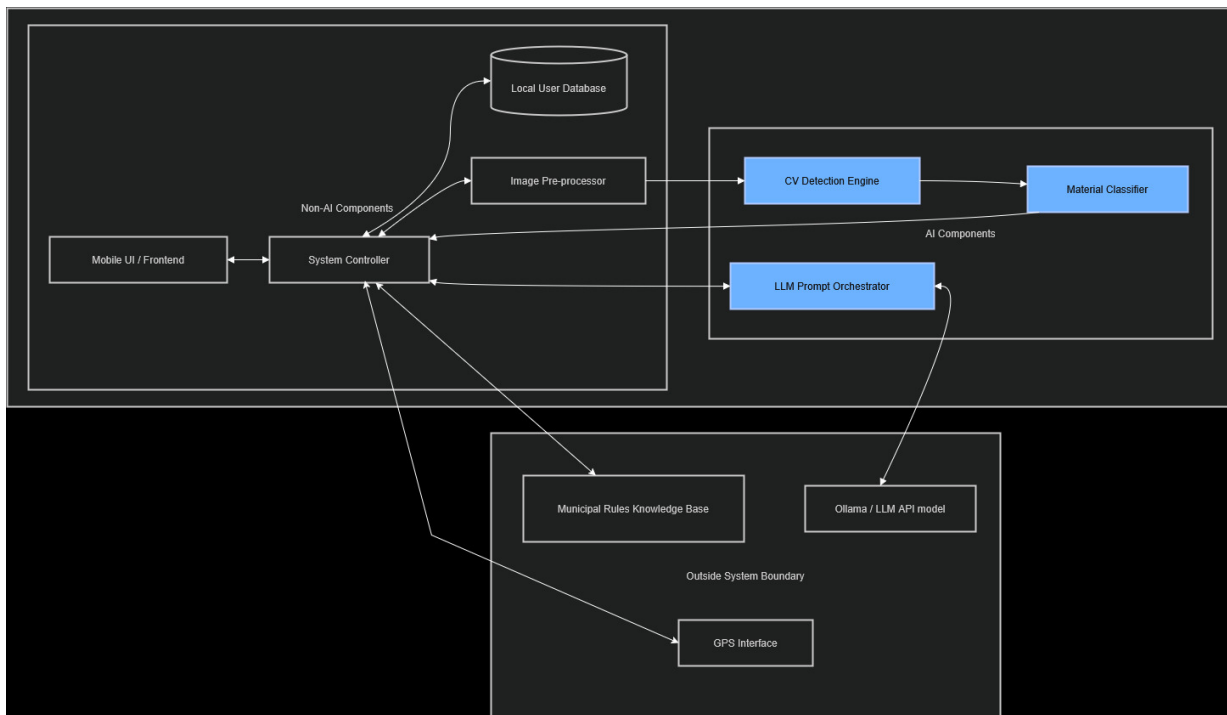
Classes and Attributes

Class	Attributes
User	userId: String, preferredLanguage: String, dataConsent: Boolean
Image	timestamp: DateTime, source: Enum(camera, gallery), rawData: ByteData
DetectedItem	itemName: String, confidence: Float, materialUncertain: Boolean
BoundingBox	x: Float, y: Float, width: Float, height: Float
Material	materialType: String, classificationConfidence: Float, category: Enum
Region	country: String, regionCode: String, gpsCoordinates: Coordinates
RecyclingRule	binColor: String, prepInstructions: List[String], depositReturn: Boolean, notes: String
ImpactStatistic	co2Saved: Float, energySaved: Float, timestamp: DateTime
AIAssistant	queryContext: String, generatedResponse: String, language: String
ScanResult	imageWidth: Int, imageHeight: Int, hasLowConfidence: Boolean, clutterWarning: Boolean

Relationships



8. Architecture Diagram



9. Component Descriptions

Mobile UI / Frontend

Files: `frontend/index.html`, `frontend/app.js`, `frontend/styles.css`

Responsibility: Single-page application served as static files by FastAPI. Handles camera capture (`getUserMedia`) and gallery upload, shows a live loading spinner during processing, renders detection results with a Certainty Meter, displays disposal instructions with step-by-step formatting, and provides a chat interface for follow-up questions. Mobile-responsive layout usable on iOS and Android.

Related requirements: Req01, Req02, Req03, Req08, Req18, NfReq01, NfReq02

System Controller

File: `backend/controller.py`

Responsibility: Orchestrates the full scan pipeline. Receives a validated PIL Image, calls the Material Classifier, maps material to a display name and recycling category, queries the Knowledge Base for local rules, calls the LLM Orchestrator for human-friendly instructions, parses the LLM output into numbered steps, calculates environmental impact stats, and persists the scan if the user has given consent.

Related requirements: Req09, Req10, Req14, Req15, Req16, Req22

Image Pre-processor

File: `backend/preprocessing.py`

Responsibility: Validates uploaded files (jpg/png only, max 15 MB), opens the image in memory, converts RGBA to RGB, and resizes so the longest side is 1024 px. The image is processed entirely in a volatile RAM buffer and is never written to disk.

Related requirements: Req24, NfReq10, NfReq16, NfReq17

Material Classifier

File: `backend/material.py`

Responsibility: Zero-shot image classification using OpenAI's CLIP (`clip-vit-base-patch32` via HuggingFace Transformers). Compares the input image against 11 recycling-specific text labels (e.g., "a plastic bottle or plastic container", "an aluminium can or metal tin") and returns the best-matching material, its confidence score, disposal category, and an `uncertain` flag if confidence is below 75%. CLIP runs on CPU; the model is pre-warmed on startup.

Related requirements: Req05, NfReq04, NfReq08, NfReq13, NfReq15

Detection Module

File: `backend/detection.py`

Responsibility: Currently a thin helper that exposes a `crop_item` utility for cropping bounding box regions from PIL images. Object detection was originally implemented with YOLOv8n (COCO 80 classes) but was removed because COCO classes do not cover most real-world waste items (cardboard boxes, tin cans, plastic bags, etc.). Material classification by CLIP on the whole image proved more accurate and reliable.

Design note: Future improvement would integrate a YOLO model fine-tuned on a waste-specific dataset (e.g., TACO) to restore bounding box detection.

LLM Prompt Orchestrator

File: `backend/llm_orchestrator.py`

Responsibility: Builds a grounded system prompt that includes item name, material, region, and the actual local recycling rules from the knowledge base. Sends the prompt to a locally running Ollama instance (Llama 3) via HTTP. Enforces a strict "recycling topics only" persona to prevent off-topic answers. Returns a structured text response or a graceful fallback message on timeout/unavailability. Chat mode maintains a rolling 6-message history window for context.

Related requirements: Req12, Req13, Req17, NfReq05

Knowledge Base

Files: `backend/knowledge_base.py`, `backend/data/rules/*.json`

Responsibility: Structured JSON files containing recycling rules per country/region and material type. Includes bin color, disposal category, preparation steps, deposit-return system flag, and notes. Supports AT (Austria), DE (Germany), KR (South Korea), and GLOBAL (general guide used as cold-start fallback). Rules are treated as pure data, so updating them never requires a code change.

Related requirements: Req15, Req22, design decision #3

Local User Database

File: `backend/database.py`, `backend/data/recycling.db`

Responsibility: SQLite database storing user preferences (language, region, data consent) and scan history (item name, material, category, region, timestamp, CO₂

saved, energy saved). No image bytes are ever stored. All history writes are gated behind explicit user consent (GDPR). History is cleared when the user revokes consent.

Related requirements: Req18, Req19, NfReq10, NfReq14

Location Module

File: `backend/location.py`

Responsibility: Resolves user location either from GPS coordinates (received from the browser's Geolocation API) or from a manually selected country code. Maps coordinates or country codes to a region identifier used as the key for knowledge base lookups. Defaults to GLOBAL when no location is set.

Related requirements: Req14, Req16, Req22, SUC2

10. Design Questions and Answers

1. How is the knowledge base kept up to date? (Req22)

The knowledge base is structured so that per-city or per-country rule updates are pure data changes, meaning you just edit a JSON file with no code changes required. The Controller reads the rules fresh on each request, so updates take effect immediately on the next scan without restarting the server.

2. What happens on cold start (no GPS and no location set)? (Req22, design decision #2)

The Controller defaults to a general **Global Recycling Guide** (`GLOBAL.json`) and prompts the user with a location-selection modal on first launch. A 500 ms delay ensures the modal appears after the UI has rendered. The user can skip the prompt and still receive general advice, or set a location to get region-specific bin colors and rules.

3. How are wrong bin colors prevented? (Req09, design decision #3)

The LLM is explicitly prohibited from inventing bin colors or local regulations. The system prompt includes the actual rules from the knowledge base under the heading "Official local rules (authoritative; base your answer ONLY on these rules)." If no rules exist for the region, the LLM is instructed to give material-based general advice and the UI explicitly states that location-specific rules are unavailable.

4. How is classification confidence communicated to the user? (Req07, design decision #4)

A color-coded **Certainty Meter** bar is shown for each detected item. The bar is green above 80%, yellow between 50–80%, and red below 50%. A warning banner appears for items below 80% confidence. The exact percentage is shown alongside the bar. This approach avoids color-only signaling (accessibility) by pairing color with numeric text.

5. How are user images handled for GDPR compliance? (NfReq10, design decision #5)

Images are validated and processed entirely in a **volatile in-memory buffer** (Python `io.BytesIO`). The PIL Image object is explicitly deleted with `del img` immediately after material classification completes, before any other processing. Image bytes are never written to disk, never logged, and never stored in the database. Only derived metadata (item name, material, category, region) is stored, and only if the user has given explicit consent.

6. What happens when items are cluttered or stacked? (Req06, design decision #6)

If more than 5 items are detected in a single scan, the system sets a `clutter_warning` flag. The UI displays a warning banner advising the user to separate items and photograph them individually. This degrades gracefully rather than returning unreliable multi-item results.

7. What happens when the material classifier is uncertain? (SUC3, design decision #7)

If CLIP's confidence for the best-matching material label is below 75%, the item is marked `material_uncertain = True`. The UI displays the material badge with a warning indicator () and presents a **"Select Material Type"** button that opens a modal with a dropdown of standard material categories. The user's manual selection overrides the AI result for that session.

8. How is the LLM restricted to recycling topics? (Req12, design decision #8)

The system prompt opens with: "You are RecycleBot, an assistant that ONLY answers questions about waste sorting, recycling, and disposal. Politely decline anything off-topic and redirect to recycling questions." Temperature is set to 0.3 (low creativity) to

reduce hallucination and off-topic drift. The system prompt is injected on every request, including follow-up chat messages.

9. How is multi-language support implemented? (Req17, design decision #9)

The user's preferred language is detected from the browser's `navigator.language` property on first load and stored in `localStorage`. It can be overridden in the Settings view. The language code is passed as a parameter to the LLM system prompt: "Respond in {language}." Llama 3 supports the major European and Asian languages required by the stakeholder list. The knowledge base rules are in English internally; the LLM translates them into the target language as part of generating the response.

10. How are overlapping bounding boxes handled? (Req21, design decision #10)

Non-Maximum Suppression (NMS) is applied by the detection layer to merge overlapping boxes for the same object. A secondary deduplication pass removes boxes with IoU (Intersection over Union) > 0.5 , keeping only the highest-confidence detection. Note: with the current CLIP-primary architecture, bounding boxes are shown as full-image overlays rather than per-item boxes, since CLIP is a classification model rather than a detection model.